Universidad de Palermo

Facultad de Ingeniería

Licenciatura en Inteligencia Artificial

Arquitectura de Computadores

Trabajo Práctico N.º 2

Pipeline, entrada/salida, interrupciones y lenguaje ensamblador

---

Integrantes

Apellido y Nombre	Legajo
Jofré, Israel Jorge Emanuel	0148948

Docente: Jorge Gabriel Echevarría

Índice

1. **¿En qué se diferencia el pipeline de un procesador básico de los 80 con uno de un procesador moderno? 3**
2. **¿Cuál es la ventaja en sistemas grandes de usar canales frente al DMA básico para E/S? 4**
3. **¿Qué es el vector de interrupciones? ¿Cómo se accede a sus entradas y qué contenido tienen sus entradas? 4**
4. **Escribir una instrucción en Assembler para sumar en la parte alta del registro AX lo que está en la posición de memoria 2000 mediante direccionamiento directo 5**
5. **¿Qué instrucciones se utilizan en debug para ver la memoria como datos y cuál como instrucciones? 5**

¿En qué se diferencia el pipeline de un procesador básico de los 80 con uno de un procesador moderno?

Un **pipeline** (cauce segmentado o vía de ejecución) divide el circuito que ejecuta el ciclo de instrucción en etapas independientes, lo que permite solapar tiempos y procesar varias instrucciones a la vez. La diferencia entre el de los años 80 y el moderno no es de grado sino de organización:

Aspecto	Pipeline básico (años 80)	Pipeline moderno
Etapas	4 a 5 (en el 80486: Fetch, Decode1, Decode2, Execute, Write-Back)	14 a 20 (hasta 31 en NetBurst, superpipelining)
Vías de ejecución	1, escalar: paralelismo solo temporal, IPC ≤ 1	varias, superescalar (ILP): paralelismo temporal y espacial, IPC > 1
Orden de ejecución	en orden	fuera de orden, con ROB
Dependencias de datos	generan burbujas	forwarding y registros de renombramiento
Saltos condicionales	predicción nula o muy simple	predicción dinámica (BTB, más del 95 %) y ejecución especulativa
Decodificación	ejecuta CISC directamente	decodifica CISC a micro-ops RISC (Front End)
Multihilo	no	sí (Hyper-Threading)

Mecanismos que incorpora el procesador moderno:

- **Superpipelining:** divide el cauce en más etapas y más cortas, lo que eleva la frecuencia de reloj.
- **Front End y Back End:** el Front End busca, decodifica (traduce cada instrucción CISC en 1 a 4 micro-operaciones RISC de tamaño fijo) y asigna recursos; el Back End ejecuta fuera de orden y retira los resultados.
- **Ejecución fuera de orden (OoO):** desde la estación de reserva despacha cada micro-operación apenas tiene sus operandos listos; el ROB (Reorder Buffer) conserva el orden original para que las escrituras finales respeten el programa.

- **Resolución de los tres conflictos del pipeline:** de datos, con forwarding y registros de renombramiento; de control, con predicción dinámica y especulación; de estructura, con más unidades funcionales (ALU, FPU, vectoriales).
- **Multihilo / Hyper-Threading:** el núcleo se presenta al sistema operativo como dos procesadores lógicos y aprovecha las burbujas ejecutando instrucciones de otro hilo.

¿Cuál es la ventaja en sistemas grandes de usar canales frente al DMA básico para E/S?

El **DMA (Acceso Directo a Memoria)** básico funciona así: el CPU lo configura con cuatro datos (origen, destino, longitud del bloque y modo de lectura o escritura), le cede el control del bus y el DMA transfiere entre el periférico y la memoria sin pasar por los registros del CPU; al terminar avisa con una interrupción. Su limitación: el CPU debe configurar el DMA en cada transferencia.

Un **canal de E/S** es un módulo mucho más independiente, con capacidad propia de procesamiento e instrucciones dedicadas a E/S. No se configura transferencia por transferencia: el CPU le ordena ejecutar un programa de E/S completo alojado en memoria y el canal lo ejecuta entero por su cuenta, interrumpiendo al CPU solo cuando todas las actividades terminaron. Hay dos variantes:

- **Canal o módulo de E/S:** procesamiento propio e instrucciones de E/S (un DMA más autónomo, que no pide permiso por cada bloque).
- **Canal procesador de E/S:** además tiene memoria propia, por lo que es un computador dedicado a la E/S, y varios pueden conectarse en cascada.

Ventaja en sistemas grandes (mainframes, grandes bases de datos): el canal **descarga casi por completo al CPU** y permite **escalar la E/S de forma independiente**. Al delegar programas de E/S enteros en vez de configurar cada transferencia, el procesador central sostiene un volumen masivo y concurrente de dispositivos sin saturarse y queda libre para procesar. En una PC esta modalidad no se usa; es propia de los grandes sistemas de datos.

¿Qué es el vector de interrupciones? ¿Cómo se accede a sus entradas y qué contenido tienen sus entradas?

Qué es. La **Tabla de Vector de Interrupción (IVT)** es una estructura ubicada al principio de la memoria principal (desde la dirección `00000h`) que asocia cada tipo de interrupción con la dirección de la rutina que debe atenderla. En el procesador 8086 tiene 256 entradas de 32 bits (4 bytes) cada una, lo que da un tamaño total de 256 × 4 bytes = 1024 bytes = 1 KB.

Cómo se accede a sus entradas. Se accede mediante el **Tipo de interrupción (Interrupt Type)**, un número de 1 byte que el dispositivo envía al CPU durante la etapa de Reconocimiento del ciclo de interrupción (en una lectura de bus posterior a la señal INTA). Al ser de 1 byte hay $2^8 = 256$ tipos posibles (00h a FFh), uno por entrada; como cada entrada ocupa 4 bytes, su dirección se obtiene como Tipo × 4:

Tipo	Direcciones que ocupa su entrada
0	`00000h` a `00003h`
1	`00004h` a `00007h`
2	`00008h` a `0000Bh`
…	…
255	`003FCh` a `003FFh`

Qué contienen sus entradas. Cada entrada almacena un puntero a la primera instrucción de la Rutina de Servicio de la Interrupción (ISR) de ese tipo. Ese puntero ocupa los 4 bytes de la entrada y está formado por dos registros de 16 bits: CS (segmento de código) e IP (puntero de instrucción). Con él, el CPU carga el contador de programa y salta a ejecutar la rutina que atiende al dispositivo.

Escribir una instrucción en Assembler para sumar en la parte alta del registro AX lo que está en la posición de memoria 2000 mediante direccionamiento directo

```
ADD AH, [2000]
```

Justificación de cada parte:

- **`ADD`:** la operación es sumar (`destino = destino + origen`).
- **`AH`** (destino): la parte alta del registro AX. AX (16 bits) se divide en dos mitades de 8 bits: AH es la parte alta (High) y AL la parte baja (Low).
- **`[2000]`** (origen): direccionamiento directo. Los corchetes indican que el operando es el contenido de la dirección de memoria 2000, no el valor literal. Sin corchetes (`ADD AH, 2000`) sería direccionamiento inmediato (sumar la constante 2000).
- La sintaxis es `MNEMÓNICO destino, origen`: lo que va después de la coma es el origen.
- En DEBUG los valores son hexadecimales, por lo que `2000` equivale a `2000h`. Como AH es de 8 bits, se suma el byte alojado en esa posición.

¿Qué instrucciones se utilizan en debug para ver la memoria como datos y cuál como instrucciones?

Para ver la memoria como **datos** se usa **`D` (Dump)**; como **instrucciones**, **`U` (Unassemble)**.

Comando	Vista	Qué hace
`D` (Dump)	Datos	Muestra el contenido en hexadecimal, con su equivalente ASCII a la derecha. Sin rango, vuelca 128 bytes. Ejemplo: `D 200 20F`.
`U` (Unassemble)	Instrucciones	Desensambla: muestra cada posición como mnemónico de instrucción del 8086, con su código de máquina. Sin rango, desensambla 32 bytes. Ejemplo: `U 0100`.